

Verifikacija softvera

Valentina Tošić

Septembar 2026

Sadržaj

1	Uvod	1
2	Testovi i pokrivenost koda	2
2.1	Pokretanje i problemi sa postojećim testovima	2
2.2	Pronađeni funkcionalni problemi	3
2.3	Dodatni testovi	3
2.4	Pokrivenost koda	3
3	Valgrind	4
3.1	Curenje memorije u klasi <code>Sto</code>	4
3.2	Curenje memorije u klasi <code>Meni</code>	4
3.3	Curenja u testovima	4
4	Clang-tidy	4
4.1	Nedostatak povratne vrednosti	5
4.2	Operator poređenja	5
4.3	Nepotrebne alokacije	5
5	Clazy	5
5.1	Klasa <code>Tabla</code>	5
5.2	Upravljanje memorijom u <code>RasporedData</code>	5
6	Fuzzing	6
6.1	Prevelike alokacije tokom deserijalizacije	6
6.2	Rušenje tokom deserijalizacije	6
6.3	Curenje memorije pronađeno fuzzingom	6
7	Heaptrack	6
8	Zaključak	7

1 Uvod

Expresso je desktop aplikacija razvijena u jeziku C++ uz korišćenje Qt6 biblioteke. Namijenjena je vođenju evidencije u kafiću i omogućava rad sa stolovima, rasporedom stolova, porudžbinama i menijem. Podaci o rasporedu i računima čuvaju se na disku.

Analizirani projekat nalazi se u Git submoduleu. Analiza je izvršena nad izvornim projektom, uz izmene koje su bile neophodne da bi projekat mogao da se prevede i testira. Originalni kod submodulea nije direktno menjan, već su potrebne izmene izdvojene u fajl `custom.patch`.

Pre početka analize bilo je potrebno rešiti nekoliko problema u samom projektu. U kodu su postojale hardkodovane apsolutne putanje do Qt biblioteka, što je onemogućavalo prevođenje u analiziranom okruženju. Takođe, funkcija `main()` postojala je istovremeno za aplikaciju i testove, što je doводilo do greške prilikom linkovanja. U test runner-u funkcija `tmain()` je morala da bude preimenovana u `main()`.

Za verifikaciju su korišćeni sledeći alati:

- Catch2 i lcov za izvršavanje testova i merenje pokrivenosti koda
- Valgrind za analizu upravljanja memorijom
- clang-tidy za statičku analizu C++ koda
- Clazy za statičku analizu Qt koda
- libFuzzer i AddressSanitizer za fuzzing i otkrivanje problema prilikom deserijalizacije
- Heaptrack za profilisanje korišćenja memorije

Za pokretanje analize prvo je potrebno inicijalizovati submodule, primeniti izmene iz fajla `custom.patch` i izgraditi projekat:

```
git submodule update --init --recursive
git apply custom.patch
./build.sh
```

Svaka analiza nalazi se u posebnom direktorijumu i ima svoju skriptu `run.sh` za pokretanje.

2 Testovi i pokrivenost koda

Za testiranje projekta korišćen je Catch2, dok je za merenje pokrivenosti korišćen lcov. Postojeći testovi nalaze se u direktorijumu `tests`, a dodatni testovi su smešteni u direktorijum `unit_tests`.

2.1 Pokretanje i problemi sa postojećim testovima

Pre samog pokretanja testova bilo je potrebno rešiti dva problema u postojećim testovima. U fajlu `tests/tmain.cpp` funkcija `main()` bila je nazvana `tmain()`, zbog čega test runner nije mogao pravilno da se pokrene. Pored toga, testovi `unesiartikle_test.cpp` i `sto_test.cpp` imali su isti naziv Catch2 test slučaja, što Catch2 ne dozvoljava. Naziv jednog testa je zato promenjen.

Nakon ovih izmena testovi su mogli da se pokrenu, ali je program završavao izvršavanje sa `SIGSEGV` nakon prva dva testa. Uzrok je bio u klasi `Porudzbina`. Konstruktor je inicijalizovao `artikli`, ali ne i pokazivač `_sto`. U funkciji `naruci` taj pokazivač se zatim koristio bez provere:

```
p->getSto()->setStatus(...);
```

Problem je rešen inicijalizacijom pokazivača na `nullptr` i proverom da li pokazivač postoji pre njegovog korišćenja. Problem je dodatno reprodukovao na originalnom kodu, bez ove izmene, čime je potvrđeno da je uzrok bio u samoj aplikaciji.

Nakon ispravke testovi su završeni sa 14 test slučajeva, od kojih je 12 prošlo, dok su 2 pala zbog stvarnih grešaka u aplikaciji.

2.2 Pronađeni funkcionalni problemi

Jedan od pronađenih problema odnosi se na računanje prihoda. Prilikom čuvanja računa ukupna vrednost se upisuje u obliku:

Ukupno: 400 RSD

Prilikom učitavanja linija se deli razmakom, nakon čega se vrednost pokušava pretvoriti u broj. Zbog tačaka koje su deo formatiranja, izdvojeni string ne predstavlja validan broj, pa `toDouble()` vraća nulu. Dodatni test je sačuvao dva računa vrednosti 250 i 150 RSD i očekivao ukupan prihod od 400 RSD. Dobijena vrednost bila je 0. Isti problem utiče na funkcije `ukupniPrihodMesec`, `ukupniPrihodGodina`, `izracunajProsecnuDnevnuZaradu`, `najprometnijiDanMesec` i `najmanjePrometanDanMesec`. Zbog toga `najprometnijiDanMesec` može vratiti nevažeći datum kada su svi prihodi jednaki nuli, dok `najmanjePrometanDanMesec` u tom slučaju može izabrati prvi dan u mesecu. Problem bi trebalo rešiti odvajanjem formatiranja od same numeričke vrednosti ili korišćenjem strukturiranog formata za čuvanje podataka.

Drugi problem postoji prilikom izmene cene artikla. Funkcija `onSubmitClicked()` prvo postavi novu cenu, a zatim pozove `dodajArtikal()`, koji dodaje novi objekat u kolekciju. Umesto izmene postojećeg artikla tako nastaje duplikat. Postojeći test ovo ponašanje registruje jer nakon izmene očekuje jedan artikal, dok se u kolekciji nalaze dva. Rešenje je da se prilikom izmene postojeći artikal samo ažurira, bez ponovnog dodavanja u kolekciju.

2.3 Dodatni testovi

Za potrebe analize dodati su testovi za funkcionalnosti koje nisu bile dovoljno pokrivene postojećim testovima:

- `izvestaj_test.cpp`,
- `kategorija_test.cpp`,
- `meni_test.cpp`,
- `binaryserializer_test.cpp`,
- `sto_extra_test.cpp`,
- `rasporeddata_extra_test.cpp`.

Postojeći test `rasporeddata_test.cpp` koristio je fiksnu putanju ka direktorijumu koji ne postoji u analiziranom projektu. Dodatni test koristi `QTemporaryDir`, čime se test izvršava u privremenom direktorijumu i ne zavisi od lokalne strukture fajlova.

Dodatni testovi su takođe otkrili da `Sto::toVariant()` koristi `m_meni->toVariant()` bez provere da li je `m_meni` validan pokazivač. Slično tome, `Meni::addItem()` može pokušati da koristi kategoriju koja ne postoji. U klasi `Meni` funkcija `deleteItem` je deklarirana, ali nije implementirana niti se koristi u aplikaciji.

2.4 Pokrivenost koda

Nakon dodavanja testova pokrivenost je povećana sa 26,2% na 40,1% za linije koda, odnosno sa 39,0% na 57,5% za funkcije. Pokrivenost grana iznosi 19,5%.

Dobijeni rezultati pokazuju da dodatni testovi značajno povećavaju pokrivenost, ali da značajan deo koda i dalje nije obuhvaćen testovima.

3 Valgrind

Valgrind je korišćen za proveru upravljanja memorijom i otkrivanje curenja memorije. Analiza je izvršena nad testovima, kao i nad osnovnim scenarijem korišćenja grafičkog interfejsa. Za GUI scenario napravljen je novi raspored, definisan je broj stolova, dodata su tri stola, raspored je sačuvan, a zatim je aplikacija zatvorena.

3.1 Curenje memorije u klasi Sto

Klasa `Sto` u destrukturu oslobađa pokazivač `_p`, ali ne oslobađa pokazivač `m_meni`. U funkciji `Sto::fromVariant()` objekat `Meni` može biti dinamički kreiran:

```
m_meni = new Meni();
```

Ukoliko se takav objekat kasnije ne oslobodi, dolazi do curenja memorije. Ovaj problem je dodatno potvrđen fuzzing analizom nad stvarno sačuvanim fajlom, gde je `AddressSanitizer` prijavio 368 bajtova u 12 alokacija koje potiču iz `Sto::fromVariant()`. Potrebno je obezbediti pravilno upravljanje objektom `m_meni` i njegovo oslobađanje kada klasa `Sto` više ne koristi taj objekat.

3.2 Curenje memorije u klasi Meni

U funkciji `Meni::deleteCategory()` objekat kategorije se uklanja iz mape, ali njegovo brisanje je zakomentarisano:

```
// delete _kategorije[naziv];
```

Objekat zato ostaje alociran. Dodatni problem postoji u klasi `Kategorija`, gde se artikli čuvaju kao sirovi pokazivači bez destruktora koji bi ih oslobodio. Potrebno je obezbediti da se ovi objekti pravilno oslobode kada više nisu potrebni. Tamo gde je moguće, korišćenje pametnih pokazivača moglo bi dodatno da pojednostavi upravljanje memorijom.

3.3 Curenja u testovima

Valgrind je prijavio i manja curenja nastala u samim testovima, gde se objekti kreiraju pomoću `new`, a zatim se ne brišu. Ova curenja se odnose na testni kod, a ne na glavnu funkcionalnost aplikacije.

4 Clang-tidy

clang-tidy je korišćen za statičku analizu svih 16 `.cpp` fajlova iz direktorijuma `Espresso/src`. Uključene su grupe provera `clang-diagnostic-*`, `clang-analyzer-*`, `modernize-*`, `performance-*`, `readability-*` i `bugprone-*`. Ukupno je prijavljeno 263 upozorenja. Većina se odnosi na stil i manje preporuke, ali nekoliko upozorenja ukazuje na potencijalne probleme u programu.

4.1 Nedostatak povratne vrednosti

Funkcija `unesiartikle::cenaArtikla` vraća vrednost unutar petlje, ali nema povratnu vrednost ukoliko traženi artikal nije pronađen. Takvo ponašanje može dovesti do nedefinisanog ponašanja. Funkciju treba dopuniti eksplicitnom povratnom vrednošću za slučaj kada artikal ne postoji ili treba prijaviti grešku.

4.2 Operator poređenja

Funkcija poređenja u klasi `Artikl` deklarirana je kao:

```
bool operator==(Artikl &a) const;
```

`clang-tidy` je prijavio da ova deklaracija može biti dvosmislena u C++20 zbog pravila o obrnutim kandidatima za operator `==`. Problem nastaje zato što je sama funkcija konstantna, dok njen parametar nije. Parametar bi zato trebalo da bude konstanta referenca:

```
bool operator==(const Artikl &a) const;
```

Na ovaj način operator može da radi i sa konstantnim objektima, a deklaracija je simetrična za oba operanda, čime se uklanja prijavljena dvosmislenost.

4.3 Nepotrebne alokacije

U funkciji `ispisiPoKategorijama()` kreiraju se objekti `QTableWidgetItem` pomoću `new`, ali je kod kojim bi oni bili ubačeni u tabelu zakomentaran. Takvi objekti ostaju alocirani i predstavljaju curenje memorije. Potrebno je ili ukloniti nepotrebne alokacije ili obezbediti njihovo pravilno oslobađanje.

5 Clazy

Clazy je korišćen za specifičnu statičku analizu Qt koda. Analiza je pokrenuta na svih 16 `.cpp` fajlova i izvršena je na nivoima 0, 1 i 2. Ukupno je prijavljeno 614 upozorenja. Najveći broj upozorenja odnosi se na nepotrebne alokacije `QString` objekata i kopiranje polimorfnih objekata. Broj upozorenja treba posmatrati uz činjenicu da se ista upozorenja mogu pojaviti više puta kada se isti kod analizira kroz različite jedinice prevođenja.

5.1 Klasa `Tabla`

Klasa `Tabla` koristi Qt mehanizam `slots`, ali nema makro `Q_OBJECT`. U trenutnom kodu povezivanje funkcija preko pokazivača na članove funkcioniše, pa ovo nije uzrok trenutno uočenog kvara. Ipak, ukoliko bi klasa u budućnosti koristila sopstvene signale, meta-objekt ili stariji Qt signal/slot mehanizam, `Q_OBJECT` bi bio potreban.

5.2 Upravljanje memorijom u `RasporedData`

Klasa `RasporedData` ručno oslobađa memoriju u destrukturu, ali nema eksplicitno definisan konstruktor kopije i operator dodele. U slučaju kopiranja objekta došlo bi

do kopiranja sirovih pokazivača, što može dovesti do dvostrukog oslobađanja memorije. Tokom analize nije pronađeno konkretno mesto u aplikaciji gde se takvo kopiranje trenutno dešava, pa se ovo smatra potencijalnim problemom dizajna, a ne potvrđenom greškom u trenutnom izvršavanju.

6 Fuzzing

Za fuzzing su korišćeni libFuzzer i AddressSanitizer. Napravljena su dva fuzzing programa. Prvi direktno testira deserijalizaciju klasa `Artikl` i `Kategorija`, dok drugi koristi `BinarySerializer::load()` i testira učitavanje celog objekta `Raspored`. Fuzzing je pokrenut pomoću posebnog CMake projekta, jer je za korišćenje opcije `-fsanitize=fuzzer` potrebno koristiti Clang.

6.1 Prevelike alokacije tokom deserijalizacije

Jedan od fuzzing testova pronašao je veoma mali ulaz koji dovodi do pokušaja ogromne alokacije memorije. Uzrok je korišćenje `QDataStream` i `QVariant` podataka bez prethodne provere veličine i strukture ulaza. AddressSanitizer je prijavio pokušaj alokacije od približno 123 GB za ulaz veličine samo nekoliko bajtova. Problem nastaje zato što se vrednosti iz neproverenog ulaza koriste kao veličine podataka koje treba učitati. Sličan problem može nastati i pri deserijalizaciji drugih Qt tipova, kao što su `QString` i `QFont`. Pre deserijalizacije potrebno je proveriti veličinu ulaza i strukturu podataka, kao i ograničiti veličine koje aplikacija može da prihvati.

6.2 Rušenje tokom deserijalizacije

Fuzzer je pronašao i ulaz od nekoliko bajtova koji dovodi do `SIGSEGV` tokom učitavanja `QBrush` objekta. AddressSanitizer pokazuje da do rušenja dolazi unutar Qt deserijalizacije, pre nego što aplikacija izvrši svoju logiku obrade dobijenog objekta. Ovo ne treba tumačiti kao grešku u Qt biblioteci, već kao posledicu toga što aplikacija prosleđuje neproverene podatke direktno Qt mehanizmima za deserijalizaciju. Rešenje je u validaciji ulaza pre deserijalizacije i proveriti da li podaci odgovaraju očekivanoj strukturi.

6.3 Curenje memorije pronađeno fuzzingom

Kada je fuzzing pokrenut nad stvarno sačuvanim fajlom aplikacije, AddressSanitizer je prijavio curenje od 368 bajtova u 12 alokacija. Trag vodi do `Sto::fromVariant()`, čime je dodatno potvrđen problem sa oslobađanjem objekta `Meni` opisan u Valgrind analizi.

7 Heaptrack

Heaptrack je korišćen za profilisanje dinamičke alokacije memorije tokom rada grafičkog interfejsa. U analiziranom scenariju maksimalna zauzetost memorije bila je približno 19,5 MB, dok je kao izgubljeno prijavljeno oko 270 KB memorije. Najveći deo prijavljenih alokacija potiče iz Qt i Wayland komponenti, posebno iz `libxkbcommon` i Qt widget komponenti. Rezultati Heaptrack-a nisu omogućili pouzdano izdvajanje svih curenja koja potiču direktno iz aplikacije, zbog čega je Valgrind korišćen kao glavni alat

za detaljnu analizu konkretnih curenja memorije. Heaptrack je ipak koristan za pregled ukupnog ponašanja aplikacije i utvrđivanje mesta na kojima dolazi do većeg broja ili većih dinamičkih alokacija.

8 Zaključak

Analiza aplikacije Espresso pokazala je nekoliko problema različitog nivoa ozbiljnosti. Najznačajniji problemi odnose se na računanje prihoda, upravljanje memorijom i obradu neproverenih podataka prilikom deserijalizacije.

Testiranjem je pronađena greška zbog koje se sačuvani prihodi učitavaju kao nula, kao i problem sa dodavanjem duplikata prilikom izmene cene artikla. Valgrind je pokazao probleme sa oslobađanjem dinamički alociranih objekata, dok su clang-tidy i Clazy ukazali na potencijalne probleme u samom C++ i Qt kodu.

Fuzzing je pokazao da aplikacija ne proverava dovoljno ulazne podatke pre deserijalizacije. Pronađeni su slučajevi u kojima veoma mali ulaz izaziva pokušaj ogromne alokacije memorije ili rušenje programa. Zbog toga bi validacija podataka pre deserijalizacije trebalo da bude jedan od prioriteta za ispravku.

Dodavanjem novih testova pokrivenost je povećana na 40,1% linija i 57,5% funkcija, dok pokrivenost grana iznosi 19,5%. Rezultati pokazuju da dodatni testovi mogu značajno poboljšati proveru postojećeg koda, ali da i dalje postoji prostor za širu pokrivenost, posebno za granične slučajeve i obradu neispravnih podataka.